MA*508 Time series: R supplement

Suppose that we observe a continuous signal in time. Let the signal be a sinusoid with a frequency of 7 Hz (7 cycles per second), see http://en.wikipedia.org/wiki/Frequency. Then we can write $x(t) = \sin(7 \cdot 2\pi t)$. For an analysis of the signal on a computer, we should consider the discrete time, specifically the time points $t_k = k/\text{FS}$ (second), $k = 1, 2, 3, \ldots$. Note that FS is called a sampling rate. If FS=1000, then $t_1 = 0.001$ sec, $t_2 = 0.002$ sec, $t_3 = 0.003$ sec and so on. For the 7Hz signal and the sampling rate FS=1000, the discrete time series consists of 1000/7=142.9 points per cycle (period).

```
install.packages("signal") # install library
library(signal) # load library
install.packages("audio") # install library
library(audio) # load library

f=7; # set the frequency 7 Hz, 7 cycles per 1 second
FS=1000; # set the sampling rate
ObsTime=3; # set the signal duration in seconds
N=FS*ObsTime; # compute the length of observed time series
t=c(1:N)/FS; # compute the time points at which the continuous signal is observed
x=sin(f*2*pi*t); # compute the signal at time moments
plot(t,x,type="l",main="signal x = sin(7 2pi t), t in (0sec,3sec)")
points(t,x,pch=19,cex=0.4)

Pgram=function(xts,title="time series") # create a function for estimating the spectrum
{    n=length(xts); I = abs(fft(xts-mean(xts)))**2/n;
P = (4/n)*I[1:(n/2)]; nu = c(0:(n/2-1))/n;
plot(nu, P, type="l", xlab="Frequency",main=title,
ylab="Scaled Periodogram"); return( cbind(nu,P) )
}
dev.new(); # create a new figure window
pgram=Pgram(x,"Periodogram of series x");

# compute the new signal as the sum of sinusoids 13Hz and 43Hz, and white noise
x2=sin(13*2*pi*t)+sin(43*2*pi*t)+rnorm(N,0,1.9);
dev.new(); # create a new figure window
plot(t,x2,type="l",main="signal x2 = sin(13 2pi t)+sin(43 2pi t)+1.9WN")
points(t,x2,pch=19,cex=0.4)
dev.new(); # create a new figure window
pgram2=Pgram(x2,"Periodogram of series x2");

fLen=100; # set the length of the filter
fPassInterval=c(30,50)/(FS/2); # set the filter spectrum as indicator function [30Hz,50Hz]
fCoef=fir1(fLen,fPassInterval,"pass"); # compute coefficients of the filter
dev.new(); # create a new figure window
freqz(fCoef,Fs=FS); # plot the filter characteristics
y2=filter(fCoef,x2); # apply filter to time series x2
dev.new(); # create a new figure window
plot(t,y2,type="l",main="y2 = filter(x2), filtering [30Hz,50Hz]")
points(t,y2,pch=19,cex=0.4)
dev.new(); # create a new figure window
pgramy2=Pgram(y2,"Periodogram of series y2");
```

```
# compute the new signal as the sum of a sinusoid with non-constant frequency and white noise
x3=sin((20+9*exp(-0.3*t))*2*pi*t)+rnorm(N,0,1.4);
dev.new(); # create a new figure window
plot(t,x3,type="l",main="signal x3 = sin((20+9exp(-0.3t)) 2pi t)+1.4WN")
points(t,x3,pch=19,cex=0.4)
dev.new(); # create a new figure window
pgram3=Pgram(x3,"Periodogram of series x3");

data(wav); # load an audio sample from the 'signal' library
FS=wav$rate; # get the sampling rate
x4=wav$sound; # get the time series of sound
t=(1:length(x4))/FS;
dev.new(); # create a new figure window
plot(t,x4,type="l",main="signal of sound, x4")
points(t,x4,pch=19,cex=0.4)
dev.new(); # create a new figure window
pgram4=Pgram(x4,"Periodogram of series x4");
play(x4,rate=FS); # play audio signal

fLen=200; # set the length of the filter
fPassInterval=c(400,600)/(FS/2); # set the filter spectrum as indicator function [400Hz,600Hz]
fCoef=fir1(fLen,fPassInterval,"pass"); # compute coefficients of the filter
dev.new(); # create a new figure window
freqz(fCoef,Fs=FS); # depict characteristics of the filter
y4=filter(fCoef,x4); # apply filter to time series x4
dev.new(); # create a new figure window
plot(t,y4,type="l",main="y4 = filter(x4), filtering [400Hz,600Hz]")
points(t,y4,pch=19,cex=0.4)
dev.new(); # create a new figure window
pgramy4=Pgram(y4,"Periodogram of series y4");
play(y4,rate=FS) # play audio signal
```

Homework.

Modify parameters of the above R script and execute it using various settings..

Observe the relationship between a time series, its filtered version, and their respective spectrums.